Datadog

Contents

8. How would you observe the OpenAI and Vertex calls: latency, cost, failures, quality?
9. What does RUM give you that backend telemetry cannot, and how does it relate to Heimdall?

Generic questions

1. What is Datadog?

Datadog is a **SaaS observability platform** that pulls metrics, logs, traces, and more into one place so you can see the health of your whole system from a single UI. It covers infrastructure, applications, and the frontend, rather than being one tool per signal.

2. What problems does Datadog solve?

Before a platform like this, each signal lived in its own tool: metrics in one, logs in another, traces in a third, and correlating an incident across them was manual and slow. Datadog solves that by **unifying the signals under one query language and one UI**, so you can jump from a spiking metric to the traces behind it to the logs for those requests without switching tools. It also handles the storage, scaling, and alerting so you do not run your own monitoring stack.

3. Why did Vendasta choose Datadog over other observability platforms?

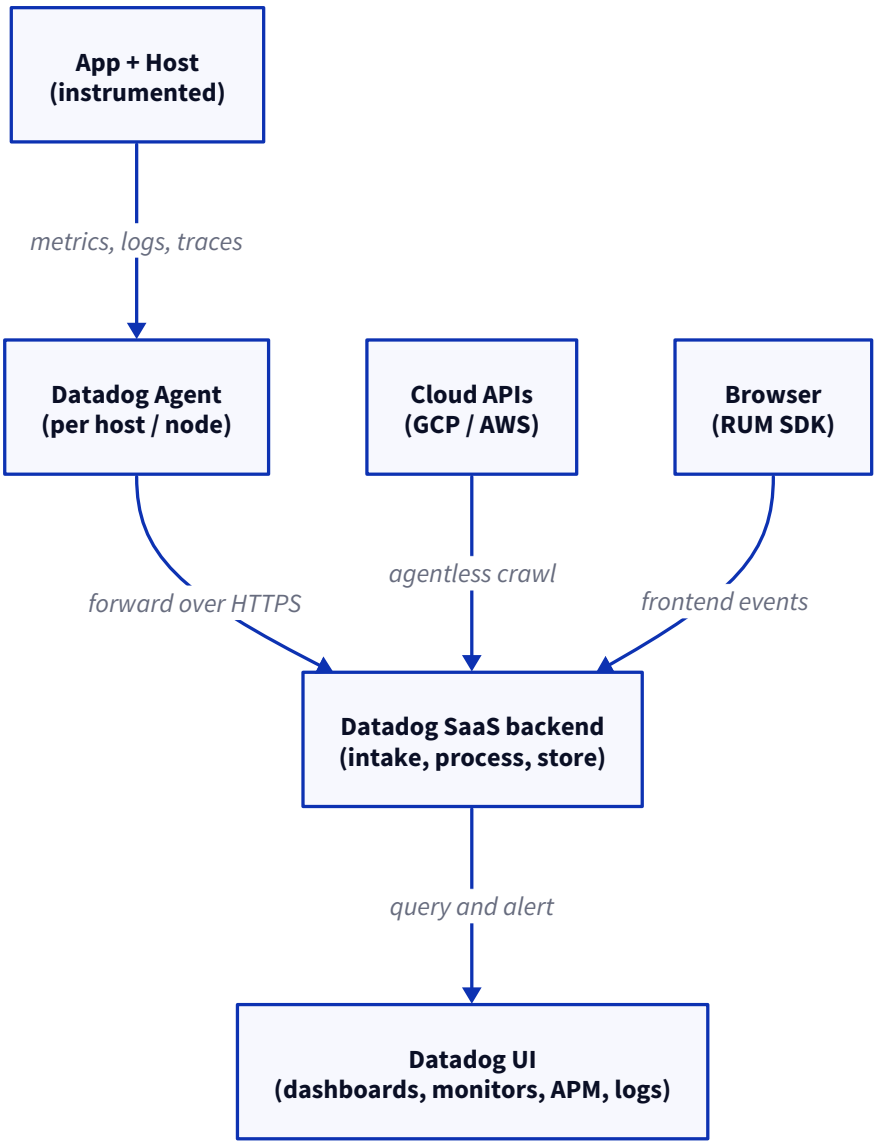
The honest answer is that the decision predates my deep involvement, so I would not claim to know the full procurement history. What I can speak to is why it makes sense for us: it is a **single platform covering metrics, APM, logs, and RUM**, so we are not stitching together Prometheus plus Grafana plus an APM tool plus a log store ourselves. It has a **first-class GCP integration**, which matters because we run on GKE. And the **SLO and monitor tooling** is strong, which is what we lean on for alerting. So the pull is mostly "one managed platform instead of running and correlating four open-source tools." *(Adapt to what you actually know of the decision.)*

4. Which Datadog products have you used?

Mostly the core four: **metrics and dashboards** for infrastructure and service health, **APM** for tracing request flows across services, **log management** for searching and correlating logs, and **monitors and SLOs** for alerting. I have also touched **RUM** on the frontend side. The deep security and profiling products I have used less. *(Adapt to your own usage.)*

5. Explain Datadog's architecture, with a diagram.

The shape is agent-based collection feeding a SaaS backend. A lightweight **Agent** runs on each host or Kubernetes node, gathers metrics, logs, and traces from the apps on that host, and forwards them over HTTPS to the **Datadog backend**, which does intake, processing, and storage. Cloud services that have no host to run an agent on (managed GCP or AWS resources) are pulled in **agentless** through their cloud APIs, and the **browser RUM SDK** sends frontend events straight to the backend. The **UI** then queries all of it for dashboards, monitors, and alerts.



6. What are the limitations of Datadog?

The big one is **cost**: it is usage-based, and custom metrics, high log volume, and APM spans add up fast, so you end up actively managing spend. Beyond that, you are **locked into a SaaS vendor** (your telemetry lives with them), high-cardinality custom metrics get expensive quickly, and the sheer number of products means a learning curve to use it well.

7. What is the Datadog Agent, how does it collect data, and how is it deployed?

The Agent is a **lightweight process that runs alongside your workloads** and ships their telemetry to Datadog.

- **How it collects**: it scrapes system metrics (CPU, memory, disk), runs **integration checks** that poll services like Postgres or Redis, tails **log files**, and receives **traces** from instrumented apps over a local port.
- **How it is deployed**: one Agent **per host** on a VM, or as a **DaemonSet** on Kubernetes so every node runs one, usually paired with a Cluster Agent.

8. Host Agent v/s Cluster Agent.

The **Host (node) Agent** runs on every node and collects that node's metrics, logs, and traces. The **Cluster Agent** is a single component that talks to the Kubernetes API **once on behalf of all nodes**, gathering cluster-level data (metadata, events, autoscaling) and handing it down to the node Agents. Without it, every node Agent would hammer the API server independently, which does not scale.

9. Can Datadog work without an Agent?

Partly. **Agentless integrations** pull data straight from a cloud provider's APIs, and the browser **RUM SDK** and some libraries report directly to Datadog. But for host-level metrics, log tailing, and APM traces you want the Agent; it is the default path for anything running on your own compute.

10. How do you troubleshoot an Agent that stops reporting data?

The way I work through it: first run `datadog-agent status` on the host, because it tells you straight away whether the Agent is running and whether each check is passing or erroring. Then I check the **basics that break silently**: is the process actually up, is the **API key** valid, and can the host **reach Datadog** (a firewall or proxy blocking the outbound HTTPS is a classic). After that I read the **Agent logs** for the failing check, and confirm the **host clock is in sync**, because bad time skews or hides the data. It is almost always one of connectivity, credentials, or a single misconfigured check rather than the whole Agent being broken.

11. What types of metrics does Datadog support?

Four core types: **Count** (how many events in an interval), **Gauge** (a value at a point in time, like memory used), **Rate** (count per second), and **Histogram/Distribution** (the statistical spread of a value, giving you percentiles like p95 and p99).

12. What are Custom Metrics, and why are they expensive?

A **custom metric** is any metric you define yourself rather than one Datadog collects out of the box, for example a business counter like orders placed. They are **billed per unique combination of metric name and tag values**, and that is the trap: a metric tagged with something high-cardinality like `user_id` or `request_id` explodes into thousands or millions of distinct series, and you pay for each. The fix is to keep tags **low-cardinality** and never tag by an unbounded id.

13. How do you create a dashboard, what widgets have you used, and how do you share them?

You build a dashboard in the UI (or as code) by adding widgets and pointing each at a metric query. The **widgets I reach for most** are the **timeseries** (a metric over time), the **query value** (a single big number like current error rate), the **top list** (worst offenders, say slowest endpoints), **heatmaps** and **distributions** for latency spread, and the **table** for per-tag breakdowns. **Sharing** is by giving teams access to the dashboard, or generating a **public/shared link** for a read-only view outside the team.

14. What is a Datadog Monitor, and what types are available?

A monitor **watches a signal and alerts when it crosses a condition** you set, so you find out about a problem before your users do. The main types:

Monitor type	What it watches
Metric	A metric crossing a threshold (CPU > 90%, error rate too high)
Log	The count or pattern of matching log events (spike in ERROR lines)
Trace Analytics (APM)	Latency or error rate on a service or endpoint from trace data
Composite	A combination of other monitors with AND/OR, to cut noise
Process	Whether a given process is running, and how many
Service Check	The status a check reports (OK / WARN / CRITICAL), like a health check
Network	Connectivity and traffic between hosts or endpoints
Database	Database health and query performance signals

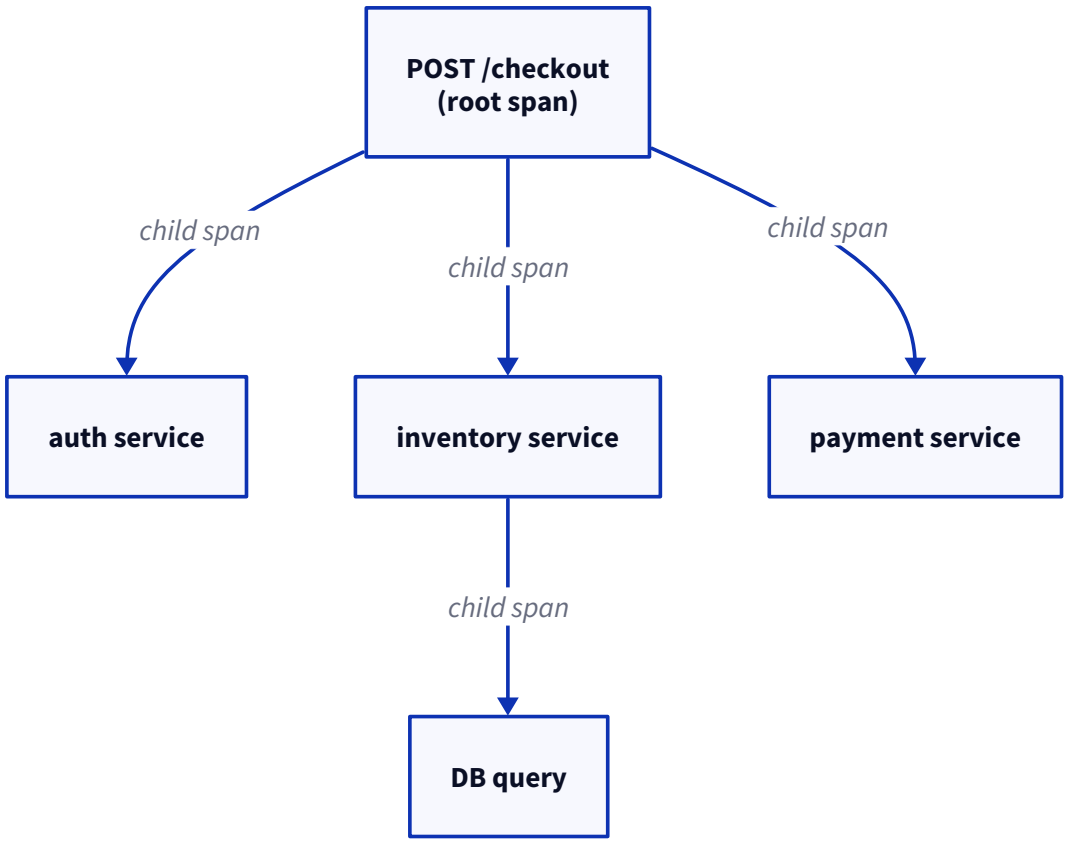
15. How does Datadog collect logs, how do you search them efficiently, and structured v/s unstructured?

The Agent (or a library) **tails and forwards logs** to Datadog, where they are parsed and indexed. To **search efficiently**, you filter by **facets and tags** (service, env, status) rather than scanning raw text, which is why tagging matters. On the last part: **structured logs** are emitted as key-value JSON (`{"status":"error","user":123}`), so the fields are queryable directly; **unstructured logs** are plain text lines that must be parsed with a pipeline before you can query the fields. Structured is far easier to search and alert on, so it is the one to prefer.

16. What is APM, and explain trace, span, and root v/s child span.

APM (Application Performance Monitoring) traces a request as it moves across your services so you can see where the time goes.

- A **trace** is the full journey of one request through the system.
- A **span** is one unit of work within that trace: a single service call, a database query, an HTTP call. Each span has a start, a duration, and tags.
- The **root span** is the first span, the entry point (the incoming request). Every other span is a **child span**, nested under a parent, and together they form a tree that shows the call hierarchy and where the latency is.



17. What is the Service Catalog, and why is it useful?

The Service Catalog is a **single inventory of all your services** with their ownership, dependencies, health, and links to dashboards and runbooks. It is useful because during an incident you can instantly see **who owns a service and what it depends on**, instead of asking around, which cuts the time to route and resolve the problem.

18. How does Datadog monitor Kubernetes, and how do the Kubernetes and GCP integrations work?

For **Kubernetes**, the Agent runs as a **DaemonSet** (one per node) plus a **Cluster Agent** for cluster-level data, and together they collect node and pod metrics, container logs, and cluster events, tagged by namespace, deployment, and pod. The **Kubernetes integration** surfaces that as pod health, resource usage, and control-plane state. The **GCP integration** is **agentless**: you connect a service account and Datadog pulls metrics from GCP's monitoring APIs for managed services (GKE, Cloud SQL, Pub/Sub), so you see them next to your own app metrics.

19. What is RUM, what is Synthetic Monitoring, and how do they differ?

RUM (Real User Monitoring) captures what **actual users** experience in the browser: **user sessions** (a full visit), **page load metrics** (load time, Core Web Vitals), **frontend error tracking** (JS errors hitting real users), and **user journey analysis** (the path users take through the app). **Synthetic Monitoring** is the opposite input: **scripted, robotic checks** that hit your endpoints or click through flows on a schedule from Datadog's own locations.

The difference is the traffic source: **RUM is passive and real** (it only sees problems your users already hit), while **Synthetic is active and simulated** (it catches problems proactively, even at 3am with no users on, but only for the paths you scripted). You use both.

20. What is the Continuous Profiler, including CPU and memory profiling?

The Continuous Profiler runs **in production continuously** to show which code is consuming resources, down to the method. **CPU profiling** tells you which functions burn the most CPU time (the hot paths to optimise), and **memory profiling** shows what is allocating the most memory (useful for chasing leaks and GC pressure). It is the step past APM: APM says which service is slow, the profiler says which line of code inside it is the cause.

21. What is Datadog Cloud SIEM?

Cloud SIEM is Datadog's **security product**: it analyses your logs against detection rules to flag security threats and suspicious activity (unusual logins, attack patterns), turning the same log pipeline you already have into security monitoring.

22. What drives Datadog cost, and how do you optimise metric, log, and APM costs?

The biggest drivers are **custom metrics** (billed per unique series), **log volume** (billed on ingest and indexing), and **APM spans** (billed on ingested and retained spans).

- **Metrics**: cut tag cardinality, drop unused custom metrics, and never tag by an unbounded id.
- **Logs**: filter at the source, and use **exclusion filters / index sampling** so you ingest cheaply but only index the logs you actually query.
- **APM**: apply **trace sampling** so you keep a representative slice (and all the error and slow traces) instead of retaining every span.

The theme across all three is the same: **control cardinality and volume at the source** rather than paying to store everything.

Vendasta-specific questions

> **Note**
>
> One thing to know up front, because it shapes every answer here: at Vendasta the signals live in different places. **Metrics** are the real Datadog surface, emitted over DogStatsD to a cluster-local Agent. **Logs** go to Google Cloud Logging. **Traces** are OpenTelemetry and export to Google Cloud Trace, not Datadog APM. So when I say "pivot to a trace", that is often a Cloud Trace step, not a click inside Datadog. I will flag that where it matters.

1. Metrics, logs, traces, RUM, and profiling: what is each for, and when a problem lands which do you reach for first?

The way I think about it: **metrics** are the always-on aggregate health (is the error rate up, is the SLO burning), **traces** show where the time or the error is for a single request across services, **logs** hold the specific error message and context, **RUM** is what the real user experienced in the browser, and **profiling** is which line of code burns CPU or memory. When a problem lands I start at the **metric**, because that is what alerts me and it tells me the shape of the problem fast. Then I pivot to a **trace** to find the hop that is slow or failing, and only then to the **logs** for that hop to read the actual error. Metrics tell me something is wrong, traces tell me where, logs tell me why.

> **Honest caveat**
>
> On our stack the honest caveat is that the trace step is in Cloud Trace and the log step is in Cloud Logging, so it is a cross-tool pivot rather than one smooth path.

Why is “publish success rate” a metric and not a log?

Because it is a **rate I want to aggregate and alert on continuously**, and metrics are built for exactly that: cheap to store, cheap to roll up into a ratio over time. In our code it literally is a metric, the `social-posts-workflow-success` and `-workflow-failure` counters tagged by network. A log is a discrete event; counting millions of them into a live success ratio is slow and expensive, and logs are not what you point an SLO at. So the outcome is a counter, and the log is there for the detail when one publish fails.

When does a trace tell you what a metric cannot?

A metric tells me **publish failures tripled**; it cannot tell me **where** in the chain (Angular to gateway to social-posts to CS to the network API) the failure is. A trace does: it breaks one request into per-hop spans, so I can see that CS is slow, or that the Facebook Graph call is the one erroring. The metric is the aggregate symptom, the trace is the anatomy of a single request. The moment the question changes from “is it bad” to “where is it bad”, I need the trace.

How does the cost and retention difference shape what you put where?

Metrics are **cheap and retained long**, so aggregate health and SLIs live there. Traces are **sampled and short-lived** (we keep about 1%), so they are for diagnosis, not for counting. Logs are the **expensive** one on ingest and indexing. That shapes a simple rule: never count things by scanning logs, emit a metric instead; keep the always-on SLI in metrics; use traces and logs for the deep dive on the small slice you actually investigate.

2. You page on SLOs and symptoms. Define the SLI, SLO, and error budget for this service, and how you turned that into monitors.

Let me be honest about what is actually wired first, then how I would define the publish one. What exists on `social-posts` today is a **generic availability SLO**: the SLI is good HTTP and gRPC status codes over total requests (an Istio-mesh query), the SLO target is set in the service’s `service-level.yaml` (for example

99.95%), and `mscli` reads that file and creates the SLO plus a burn-rate alert in Datadog. That covers all traffic, not publishing specifically.

If I were defining the **publish** SLO, I would build the SLI on the counter we already emit: **successful publishes over total publishes**, split by network. The SLO might be 99% over a rolling 30 days, and the **error budget** is the 1% you are allowed to fail, roughly the number of failed publishes you can absorb in the window before you have to stop and fix reliability. The monitor is a burn-rate alert on that SLI. We do not have this publish-specific SLO in code today, so I would flag that as the gap and the workflow-success/failure counter as the thing to build it on.

Symptom v/s cause alerting: why did the shift cut false pages?

Paging on a **symptom** (the SLO is burning, users cannot publish) means I only get woken when something users actually feel is broken. Paging on a **cause** (CPU is high, a pod restarted, a queue is deep) fires constantly on things that self-heal or do not matter to anyone, so you get 3am pages for a blip that recovered on its own. Moving the page to the symptom cut the noise because most causes never become a symptom.

How do you pick the target and the window, rolling v/s calendar?

The target comes from **what users need and what you actually deliver now**, not a round number; setting 99.99% when you run at 99.9% just guarantees a burning budget. The **window** is rolling v/s calendar: **rolling** (always the last 30 days) is steadier and there is no artificial reset, **calendar** (resets on the 1st) lines up with business reporting but gives you a cliff where the budget suddenly refills. Our `mscli` SLOs run on 7, 30, and 90 day timeframes. I lean rolling for paging because the budget reflects reality continuously.

Fast-burn and slow-burn alerts, and what you do when the budget is spent.

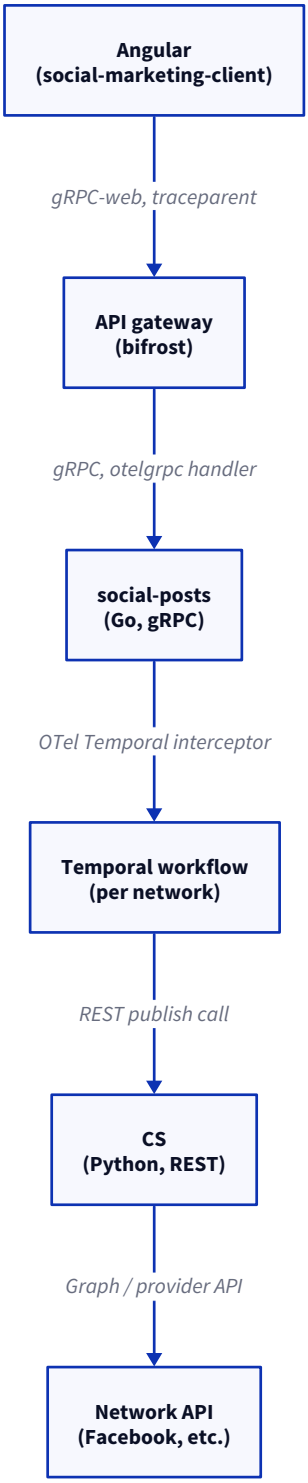
Fast-burn watches a short window (we use 5 minutes against 1 hour, alerting when you would consume 5% of the budget in an hour) and pages, because that pace blows the budget in a day. **Slow-burn** watches a long window (say 6 hours) for a slow leak and should open a ticket, not page.

> **Honest gap**
>
> `mscli` only generates the single fast-burn window; I did not find a slow-burn companion in code, so that is something I would add.

When the **budget is spent**, the budget is a decision rule, not just a number: you stop shipping risky changes and put the effort into reliability until you are back in budget.

3. A publish request flows Angular to social-posts to CS to a network API. How do you trace it end to end, and how does trace context propagate across the gRPC hops?

The flow is browser to API gateway (bifrost) to social-posts, which runs a Temporal workflow per network, out to CS over REST, and finally to the provider's API.



Our tracing is **OpenTelemetry**, and context rides the gRPC hops through the **otelgrpc stats handler**: it writes the W3C `traceparent` into gRPC metadata on the way out, and the receiving server extracts it and continues the same trace. That server-side handler is on by default for every Vendasta gRPC service. Two honest caveats. First, the plain service-to-service client (the `vax` dialer most calls use) still carries the older OpenCensus header, bridged into OTel during a migration, so it is not uniformly the W3C header everywhere yet. Second, the CS hop is Python over REST, so trace continuity there depends on the header being propagated across that boundary. And the whole thing exports to **Google Cloud Trace**, so “trace it end to end” is a Cloud Trace exercise, while the metrics and logs sit elsewhere.

How do trace and span IDs cross a gRPC boundary and get into a Temporal workflow?

On gRPC, the otelgrpc handler **injects the traceparent into gRPC metadata** and the server extracts it, so parent and child spans link across the wire. For Temporal, we register the **official OpenTelemetry Temporal interceptor** on the client, which carries the span context into the workflow, so the workflow and its activities join the same trace instead of starting a fresh one.

Head v/s tail sampling, and what you lose with each.

Head sampling decides at the start of the request whether to keep the trace. It is cheap, and it is what we do (about 1%, with a guaranteed minimum throughput so low-traffic services still get some traces). The loss is that you decide **before you know the outcome**, so you can drop the very error trace you needed. **Tail sampling** decides after the request finishes, so you can keep all the errors and slow ones, but it needs a collector buffering every span, which is more infrastructure. We are head-based, so the trade is cost for the occasional missed error trace.

Keeping a trace intact across an async Pub/Sub hop.

> **Honest answer**
>
> On our stack a **Pub/Sub hop breaks the trace**. Nothing injects trace context into the message attributes at the SDK level, so the publisher span and the subscriber span are not linked. To keep it intact you inject the `traceparent` into the message attributes when publishing and extract it on the subscriber, joining them with a span link. Today that is a gap, so a flow that crosses Pub/Sub shows up as two disconnected traces.

4. Datadog cost climbs with custom metrics, indexed logs, and ingestion volume. How do you keep it down without going blind?

The usual culprit for us is **custom-metric cardinality**, and it is not hypothetical. We have counters tagged by unbounded ids: `account_group_id` on reputation workflow counters, `partner_id` on account activations, `sub_user_id` on an email path, `account_group` on the image-generation metric. Datadog bills a custom metric per unique combination of name and tag values, so tagging by an unbounded id turns one metric into as many billed series as there are account groups or partners hitting that code. Keeping signal without going blind means **aggregating at the right tag level**: keep low-cardinality tags like `network`, `env`, `service`, and never tag a metric by an id.

Indexing v/s archiving logs, and index filters or sampling.

Indexing makes logs searchable and is the expensive part; **archiving** drops them cheaply into cold storage you can rehydrate later. The move is to index only what you query live (errors, the key paths) and archive the rest, with exclusion filters trimming the noise before it is indexed.

> **Honest flag for us**
>
> Our application logs go to **Google Cloud Logging**, and I did not find a Datadog log pipeline in code, so these index and archive knobs, if logs reach Datadog at all, would be managed in the Datadog UI or an infra repo rather than in our service code.

Custom-metric cardinality as the usual culprit: how do you find it?

Datadog's **metrics summary** shows the number of timeseries behind each metric. Sort by that, and the metric with a runaway count is the one with an unbounded tag. Open it, look at the tags, and it is almost always an id like `account_group_id` that is doing the damage.

Log-to-metric to keep the signal cheaply, and rehydration trade-offs.

If all I need from a log is a **count** (how many 5xx per minute), I generate a metric from the log at ingest and stop indexing the raw lines, so I keep the trend for almost nothing. **Rehydration** is pulling archived logs back into a searchable index when I need to investigate a specific window; the trade-off is that it is slower and you pay to rehydrate, so it suits the occasional deep dive, not routine searching.

5. The publish success rate SLO is burning. Walk me through using Datadog to find the cause fast.

First I look at the **workflow-failure counter split by the `network` tag**, because that immediately tells me whether it is every network or just one. If it is one, I already suspect the provider. Then I grab an exemplar failing request and open its **trace** to see which hop is erroring (usually the CS to network API call), and jump to that hop's **logs** for the exact error. In parallel I check the **deploy marker**: we push a deploy event to Datadog on every `mscli` deploy and it is overlaid on the error graphs, so I can see at a glance whether a release lines up with the spike. Between the network split and the deploy marker I usually know within minutes whether it is us or them.

Pivoting from the monitor to traces to logs on the same request.

From the burning monitor I take one failing request, open its trace to find the failing span, then pivot to that span's logs for the error message.

> **Honest caveat again**
>
> Our traces are in Cloud Trace and logs in Cloud Logging, so this pivot crosses tools rather than being one click inside Datadog, and I lean on a shared request id to line them up.

Filtering by network tag to see whether it is one provider.

The success and failure counters are tagged `network`, so I split the failure count by that tag. All the failures being `facebook` and the other networks flat is the fastest possible signal that this is a Facebook problem, not a regression in our shared publish code.

Separating a real regression from a network's own outage, and correlating with a deploy marker.

If failures spike for a single network with **no deploy near that time**, it is very likely the provider, so I check their platform status. If the spike lines up with **our deploy marker** on the graph, it points at our release, and I look at what changed. That deploy overlay is exactly the thing that separates "we broke it" from "they are down" quickly.

6. How do you design a monitor that catches real problems without paging on noise, and which monitor types do you actually use?

The principles: page only on **symptoms** users feel, match the **condition** to the signal, add an **evaluation window** and a separate **recovery** threshold so it does not flap, and attach **routing and a runbook** so the page is actionable. A page that fires on a single spiky data point, or with no next step attached, trains people to ignore it.

Threshold v/s anomaly v/s forecast v/s composite.

- **Threshold**: a fixed line (error rate above 5%). Best when you know the number.
- **Anomaly**: alerts on deviation from the normal pattern. Right for seasonal metrics where a fixed line is wrong at 3am v/s noon.
- **Forecast**: alerts before you hit a limit (disk will fill in three days). Right for slow trends.
- **Composite**: combines monitors with AND/OR to cut noise (page only if error rate is high AND traffic is normal, so you ignore the error rate when nobody is calling).

Evaluation window and recovery conditions to stop flapping.

I require the condition to hold for a **window** (a few minutes) rather than firing on one data point, and I set a **recovery threshold** below the trigger so it does not toggle alert-and-recover on the boundary. Those two together kill most flapping.

Alert grouping, routing, and runbook links so the page is actionable.

I **group by a tag** (per network, per service) so one incident is one alert instead of fifty, **route** it to the owning team, and **link a runbook** so whoever is paged knows the first three steps. Our `mscli`-generated SLO alerts are already tagged with the service owner, which is the hook the routing hangs on.

7. Tell me about an incident where Datadog was central to detection or diagnosis, and what you changed afterward.

A representative one: publish failures started climbing for a single network, and the **burn-rate alert on the availability SLO** caught it before anyone reported it. I split the failure counter by the `network` tag and saw it was isolated to one provider, opened a trace and found the CS to provider call timing out, and read the provider's error code in the logs. There was **no deploy marker** near the spike, so it was the provider degrading, not us. Afterward I added a **per-network publish-failure monitor**, so next time a single provider goes bad we catch it directly instead of waiting for the overall SLO to burn. *(Adapt this to a real incident you owned; keep the signal-to-diagnosis-to-fix shape.)*

Which signal caught it first: SLO burn, error tracking, or a user report?

The **SLO burn-rate alert**, which is the whole point of symptom-based alerting: you find out from your own signal before a customer opens a ticket.

What the correlation across metrics, traces, and logs revealed.

The **metric** said failures were up, splitting by the **network tag** isolated it to one provider, the **trace** showed the failing hop, and the **logs** gave the provider's specific error code. Each layer narrowed it further.

The fix, and the new monitor or dashboard that came out of it.

Short term, lean on the workflow retry and fail the post to a draft rather than losing it. Long term, the **per-network failure monitor** so a single degrading provider pages us on its own, without hiding inside the aggregate.

8. The product calls OpenAI and Vertex for captions and images. How would you observe those calls: latency, cost, failures, quality?

Let me correct the premise from what is actually in the code, then answer. **Captions** go to OpenAI (GPT-4.1) through our shared `gosdks/openai` client, and that client already emits solid per-model telemetry: latency, token counts, and success and error counts, all tagged by `model`. **Images** are generated in social-posts, either GPT-Image-2 on OpenAI or `gemini-3.1-flash-image` on Vertex, and that path is thin: a single success counter tagged model, quality, and account group, with no latency, no tokens, and no error metric. There is **no video generation** at all (video is user-uploaded, not AI-made), and there is **no dollar-cost metric anywhere**, only token counts, and we do **not** use Datadog LLM Observability.

> **Honest state**
>
> Text is well observed, images are barely observed, and cost and quality are not measured.

Token usage and cost per generation as a first-class metric.

On the text path we already emit **token counts per model**, which is the raw input to cost. We do not turn that into a **dollar figure** anywhere. I would add a cost metric derived from tokens times the per-model price, so spend is a first-class number rather than something you reconstruct from the bill.

Latency and error rate per model and provider.

The text path has both, tagged by model. The **image path has neither**, just a success count, so a slow or failing image model is invisible. I would add latency and error counters to the image path so Gemini and GPT-Image-2 are comparable side by side.

Tracing a fallback from Gemini to the OpenAI image model, and what Datadog can and cannot tell about output quality.

> **Honest correction**
>
> There is **no such fallback** in the image path. The provider is chosen by a fixed model-to-provider map, and if Gemini errors it returns the error rather than retrying on OpenAI (the cross-vendor fallback that exists lives in a separate AI-assistants chat product, not here). So there is nothing to trace there today.

On **quality**: Datadog can tell me latency, errors, and tokens, but it **cannot judge whether the caption or image is good**. Output quality needs evals or user signals (regenerate rate, thumbs, edits before posting), not APM.

9. The frontend is an Angular app in galaxy. What does RUM give you that backend telemetry cannot, and how does it relate to error tracking like Heimdall?

The honest headline: **we do not run Datadog RUM**. Frontend errors go to **Heimdall**, which is Vendasta's own error-monitoring service; the Angular apps register an error handler that posts every uncaught error to the Heimdall API. So the thing RUM would give that backend telemetry cannot, real-user page load, the actual browser experience, session context, is largely a **gap** for us. Heimdall covers the error slice (with stack traces and the deployment it happened on, which is genuinely useful for catching a bad deploy), but not performance or session data. Backend telemetry only ever sees server time; it never sees the render, the user's network, or a JavaScript error that never reached the server.

Core Web Vitals and real-user latency v/s synthetic checks.

RUM measures what **real users** actually experience (their load times, LCP, CLS from real devices and networks), while synthetic checks are scripted probes from fixed locations. Backend metrics see neither.

> **Honest state**
>
> Because RUM is not wired, we do not have Core Web Vitals or real-user latency today; that is a gap I would close if frontend performance became a priority.

Session replay and privacy in a multi-tenant product.

Session replay records the real session so you can watch a bug happen. In a **multi-tenant** product the hard part is privacy: you must mask inputs and PII and set a strict default privacy level, so you never leak one partner's data into a replay. We do not run session replay, so this is about how I would approach it rather than what we do.

Linking a frontend RUM error to the backend trace that caused it.

With RUM you can stitch a browser error to the backend trace (via the allowed tracing origins config), so you jump straight from the user's error to the server span that failed. For us this is **not wired**: Heimdall captures the frontend error with its stack and deployment, but it does not link to a backend trace, so that correlation is manual today, usually lining things up by timestamp and a shared request id.